Universidad Carlos III

Heurística y Optimización

Práctica 1: Programación Lineal Entera con GLPK

Curso 2026-27

Grupo: 01

Julio César Ruiz Sánchez (NIA: 100522170)

Denis Ángel Sánchez Rybianska (NIA: 100550260)

1. Introducción

En este lab estamos ante un problema de palets que se debe resolver con GLPK y un script de python, la memoria está estructurada en estas partes:

- Parte 1: Ordenamiento de cajas en un nivel de dos dimensiones.
- Parte 2: Expansión a tres niveles, introduciendo peso y capacidad de carga.
- Parte 3: Generalización que permite palets incompletos y espacios vacíos.
- Parte 4: Extensión final a un escenario con múltiples palets disponibles y costes preestablecidos.
- Parte 5: Análisis de resultados que evalúa características como la complejidad teórica, simetría, rendimiento computacional y posible infactibilidad de las aproximaciones.

En la memoria se emplea notación algebraica, mientras que en los ficheros .mod los conjuntos y parámetros llevan nombres descriptivos: K → CAJAS, I → FILAS, J → COLUMNAS, M → NIVELES, p_k → prioridad, w_k → peso, γ_k → capacidad, Γ→Gam, S→PALETS, κ→coste_palet, p^∞→prioridad_inf y β→beta.
Cada script escribe la disposición óptima en un fichero de texto con el mismo nombre y ruta que el fichero de datos,cambiando su extensión por `.out` (o por .sol si el fichero de datos ya termina en .out).

2. Parte 1: un palet, un nivel

Queremos minimizar el coste medio de la disposición de las cajas sobre un único palet de lado l con un solo nivel. Estamos ante un problema de programación lineal entera mixta: las variables de asignación son binarias y la variable auxiliar de prioridad es continua. Es un problema de asignación -n cajas en n huecos

2.1. Conjuntos y parámetros

Parámetros:

$l \in \mathbb{Z}$, $l \geq 1$ lado del palet

$p_k \in \mathbb{Z}$, $p_k \geq 1$ prioridad de la caja k (menor valor, mejor prioridad)

conjuntos:

K = (1..n): cajas, con n = l al cuadrado

I = {1, …, l} filas; i = 1 es el fondo, i = l el lado del acceso
J = (1..l) columnas

(1,1) es la esquina inferior izquierda, el primer índice representa la fila y el segundo la columna

2.2. Variables de decisión

V1: $x[k,i,j] \in \{0,1\} \ \forall \ k \in K, i \in I, j \in J$

V2: $P[i,j] \geq 0 \ \ \forall \ i \in I, j \in J$

x decide la disposición: recorre todos los emparejamientos caja-hueco posibles, de modo que una solución entera es una selección de n de ellos.

P queda determinada por x mediante R4; su función es dar nombre a la prioridad de cada hueco para poder escribir el orden y el coste sin arrastrar el sumatorio sobre k. En total $l^2(n+1)$ variables, de las cuales $n \cdot l^2$ son binarias.

2.3. Función objetivo

El coste de una sola caja en (i,j) es la suma de las prioridades de las cajas que tiene por delante en su misma columna. El bloqueo solo se da dentro de una columna, las cajas de la misma fila no se estorban, no se especifica pero es lógico y se ve en la figura 3 del enunciado.

$c(i,j) = \Sigma_{i'=i+1..l} P[i',j]$

sumando sobre todos los huecos y dividiendo entre el número de cajas:

$\min z = (1/n) \cdot \Sigma_{i \in I} \Sigma_{j \in J} \Sigma_{i'=i+1..l} P[i',j]$

cada P[i,j] aparece una vez por cada fila que tiene detrás, es decir $(i-1)$ veces.

$$\min z = (1/n) \cdot \Sigma_{k \in K} \Sigma_{i \in I} \Sigma_{j \in J} (i-1) \cdot p_k \cdot x[k,i,j]$$

$(i-1)$ es el número de huecos que pagan por esa caja.(son dos maneras de representar la función ahora usamos la primera). La expresión es válida porque con el palet lleno todos los huecos tienen una caja.

2.4. Restricciones

R1: $\Sigma_{k=1..n} x[k,i,j] = 1 \ \forall \ i \in \{1..l\}, \ \forall \ j \in \{1..l\}$

Cada posición tiene exactamente una caja, sin huecos vacios genera l^2 restricciones. n=l^2

R2: $\Sigma_{i=1..l} \Sigma_{j=1..l} x[k,i,j] = 1 \ \forall \ k \in \{1..n\}$

Cada caja está en exactamente una posición genera n restricciones. n=l^2

R3: $P[i,j] \geq P[i+1,j] \ \forall \ i \in \{1..l-1\}, j \in J$

Condición de que cada caja se ubique por delante de las que tienen menos prioridad que ellas. es >= para así admitir prioridades repetidas, el índice i llega solo hasta $l-1$ para no referenciar la fila l+1". Genera $l(l-1)$ restricciones.

R4: $P[i,j] = \Sigma_{k \in K} p_k \cdot x[k,i,j] \ \forall \ i \in I, j \in J.$

teniendo en cuenta la restricción de la variable que hemos nombrado como auxiliar p, como en cada hueco solo hay una caja con x=1, el sumatorio sólo muestra la prioridad de esa caja, genera l^2 restricciones

En total $2l^2 + n + l(l-1)$ restricciones

2.5. Decisiones de diseño

Decisión de diseño sobre la variable P

la variable P la hemos definido como una variable auxiliar, esta toma valores enteros automáticamente al ser las x binarias, por lo que se declara como variable continua para esto se necesita una restricción que es la propia fórmula, ya que sino P[i,j] sería una variable libre y el programa podría asignarle cualquier valor que obtuviera un coste óptimo.

Esto genera l ^2 restricciones y l^2 variables adicionales .El coste de tiempo en el programa no cambia prácticamente nada, pero al separar el problema con variables se hace más fácil su explicación

3. Parte 2: varios niveles, pesos y capacidades

Se amplía la parte 1 ahora pueden haber h niveles de cajas sobre el palet, las cajas ahora tienen peso y capacidad de peso que aguantan. Se trata de un problema de programación lineal entera mixta: las variables x son binarias, y P, W y Γ son continuas, aunque toman valores enteros porque se definen como combinaciones de binarias con coeficientes enteros. Muchos aspectos los mantenemos igual de la anterior parte

3.1. Conjuntos y parámetros

Conjuntos:

K = (1..n): cajas, con $n = h \cdot l^2$

I = (1..l): filas (i = 1 es el fondo, i = l la fila de acceso)

J = (1..l): columnas

M = (1..h): niveles (m = 1 es el inferior, m = h el superior)

Parámetros:

$l \in \mathbb{Z}$, $l \geq 1$ (lado del palet) ; $h \in \mathbb{Z}$, $h \geq 1$ (número de niveles);

$p_k \in \mathbb{Z}$, $p_k \geq 1$ prioridad de la caja k (menor valor, mejor prioridad)

$w_k \geq 0$(peso de la caja k) ; $\gamma_k \geq 0$ peso máximo que la caja k soporta encima, en gramos

3.2. Variables de decisión

V1: $x[k,i,j,m] \in \{0,1\} \quad \forall\, k \in K, i \in I, j \in J, m \in M$

Vale 1 si la caja k ocupa el hueco de la fila i, columna j y nivel m, y 0 en caso contrario,genera $n \cdot l^2 \cdot h = h^2 \cdot l^4$ variables.

V2: $P[i,j,m] \geq 0 \;\forall\, i \in I, j \in J, m \in M$

es la variable de prioridad de una caja en el hueco (i,j,m) $h \cdot l^2$ variables

V3: $W[i,j,m] \geq 0 \ \forall \ i \in I, j \in J, m \in M$

Muestra el Peso de una caja en el hueco(i,j,m)$h \cdot l^2$ variables

V4: $\Gamma[i,j,m] \geq 0 \ \forall \ i \in I, j \in J, m \in M$

Muestra la capacidad de una caja en el hueco(i,j,m)$h \cdot l^2$ variables

En total $h \cdot l^2(n+3)$ variables, de las cuales $n \cdot h \cdot l^2$ son binarias.

3.3. Función objetivo

El coste de la caja del hueco (i,j,m) es la suma de las prioridades de las cajas que tiene por delante en su misma columna y nivel. Es el mismo coste de la parte 1 con el índice de nivel añadido,con $n = h \cdot l^2$:

$$\min z = (1/n) \cdot \Sigma_{k \in K} \Sigma_{i \in I} \Sigma_{j \in J} \Sigma_{m \in M} (i-1) \cdot p_k \cdot x[k,i,j,m]$$

El coeficiente $(i-1)$ sigue siendo válido porque el palet también va lleno, el coste no depende del nivel ya que solo de pende de fila columna en cada nivel, y los niveles son independientes

3.4. Restricciones

(R1) Cada hueco, una caja ($l^2 \cdot h$ restricciones)

$$\Sigma_{k \in K} x[k,i,j,m] = 1 \quad \forall \ i \in I, j \in J, m \in M$$

Es una igualdad porque el palet va lleno: ningún hueco puede quedar vacío ni tener más de una caja.

(R2) Cada caja, un hueco ($h \cdot l^2$ restricciones)

$$\Sigma_{i \in I} \Sigma_{j \in J} \Sigma_{m \in M} x[k,i,j,m] = 1 \quad \forall \ k \in K$$

Misma restricción que en la parte 1 pero con la ampliación de nivel m

(R3) Orden por prioridad ($l \cdot (l-1) \cdot h$ restricciones)

$$P[i,j,m] \geq P[i+1,j,m] \quad \forall \ i \in \{1..l-1\}, j \in J, m \in M$$

Misma restricción que en la parte 1 pero con la ampliación de nivel m, el cual queda libre ya que la prioridad es independiente en cada nivel

(R4) prioridad de hueco

$$P[i,j,m] = \Sigma_{k \in K} p_k \cdot x[k,i,j,m] \quad \forall \ i \in I, j \in J, m \in M$$

genera $l^2 \cdot h$ restricciones

(R5) Peso de una caja en un hueco

$$W[i,j,m] = \Sigma_{k \in K} w_k \cdot x[k,i,j,m] \quad \forall \ i \in I, j \in J, m \in M$$

genera $l^2 \cdot h$ restricciones

(R6) Capacidad de aguantar peso de una caja en un hueco

$$\Gamma[i,j,m] = \Sigma_{k \in K} \gamma_k \cdot x[k,i,j,m] \quad \forall \ i \in I, j \in J, m \in M$$

genera $l^2 \cdot h$ restricciones

(R7)Resistencia acumulada ($l^2 \cdot (h-1)$ restricciones)

$$\Sigma_{m'=m+1..h} W[i,j,m'] \leq \Gamma[i,j,m] \quad \forall\ i \in I,\ j \in J,\ m \in \{1..h-1\}$$

el peso soportado, sumando todas las cajas de encima en la misma fila y columna, es menor o igual que la capacidad. Es lineal porque ambos lados son combinaciones lineales de las variables. m= h no pasa nada porque al estar arriba no tiene peso que aguantar

3.5. Decisiones de diseño

Variables auxiliares

Igual que se hizo con P en la parte 1, se introducen variables continuas indexadas por hueco que dan el valor de lo que haya en (i,j,m), el peso prioridad y capacidad.Son iguales porque las auxiliares quedan completamente determinadas por x. No amplían el espacio de búsqueda. Generan $3 \cdot l^2 \cdot h$ variables y $3 \cdot l^2 \cdot h$ restricciones.

4. Parte 3: palets incompletos

Esta parte implementa el caso $n \leq h \cdot l^2$, en el que pueden quedar huecos vacíos. Seguimos ante un problema de programación lineal entera mixta: la familia de asignación x es binaria y el resto de variables son continuas, determinadas por sus igualdades de definición. Estructuralmente es un problema de asignación con restricciones de secuenciación

4.1. Conjuntos y parámetros

Conjuntos:

$K = \{1..n\}$: cajas, con $1 \leq n \leq h \cdot l^2$

$I = \{1..l\}$: filas (i = 1 es el fondo, i = l la fila de acceso)

$J = \{1..l\}$: columnas

$M = \{1..h\}$: niveles (m = 1 es el inferior, m = h el superior)

Parámetros:

$l \in \mathbb{Z}$, $l \geq 1$ (lado del palet) ; $h \in \mathbb{Z}$, $h \geq 1$ (número máximo de niveles)

$n = |K|$, $1 \leq n \leq h \cdot l^2$ (número de cajas)

$p_k \in \mathbb{Z}$, $p_k \geq 1$ prioridad de la caja k (menor valor, mejor prioridad)

$w_k \geq 0$ (peso de la caja k, en gramos) ; $\gamma_k \geq 0$ peso máximo que la caja k soporta encima, en gramos

Parámetros derivados, calculados dentro del modelo a partir de los datos para no romper la generalidad:

$p^{max} = \max_{k \in K} p_k$, peor prioridad de la instancia

p^∞ = p^max + 1, valor finito con el que representamos el +∞ del hueco vacío

β_i = (l − i)·p^max, i ∈ {1..l−1}, cota superior del coste que puede acumular un hueco de la fila i

4.2. Variables de decisión

V1: x[k,i,j,m] ∈ {0,1} ∀ k ∈ K, i ∈ I, j ∈ J, m ∈ M

Vale 1 si la caja k ocupa el hueco (i,j,m), 0 en caso contrario. n·h·l² variables.

V2: y[i,j,m] ∈ [0,1] ∀ i ∈ I, j ∈ J, m ∈ M

Ocupación del hueco: 1 ocupado, 0 vacío. h·l² variables.

V3: P[i,j,m] ≥ 0 ∀ i ∈ I, j ∈ J, m ∈ M

Prioridad a efectos de coste: p_k si hay caja, 0 si el hueco está vacío. h·l² variables.

V4: Q[i,j,m] ≥ 0 ∀ i ∈ I, j ∈ J, m ∈ M

Prioridad a efectos de orden: p_k si hay caja, p^∞ si el hueco está vacío. h·l² variables.

V5: W[i,j,m] ≥ 0 ∀ i ∈ I, j ∈ J, m ∈ M

Peso de la caja alojada, 0 si el hueco está vacío. h·l² variables.

V6: Γ[i,j,m] ≥ 0 ∀ i ∈ I, j ∈ J, m ∈ M

Capacidad de la caja alojada, 0 si el hueco está vacío. h·l² variables.

V7: c[i,j,m] ≥ 0 ∀ i ∈ {1..l−1}, j ∈ J, m ∈ M

Coste puesto al hueco, 0 si está vacío. h·l(l−1) variables. No se declara en la fila de acceso

En total h·l²(n+5) + h·l(l−1) variables, de las cuales n·h·l² son binarias. Solo x se declara entera: y, P, Q, W , Γ y c quedan forzadas a valores enteros por sus igualdades de definición.

4.3. Función objetivo

$$min\ z = (1/n) \cdot \Sigma_{i=1..l-1} \Sigma_{j\in J} \Sigma_{m\in M} c[i,j,m]$$

El coste de una caja es la suma de las prioridades de las cajas que tiene por delante en su misma columna y nivel. Como R3 obliga a que los huecos ocupados de cada columna-nivel formen un bloque contiguo pegado al acceso, por delante de una caja no hay nunca huecos vacíos, de modo que la suma de P sobre i' = i+1..l es exactamente la suma de las prioridades que la bloquean. Para un hueco vacío el coste debe ser 0(R10)

4.4. Restricciones

(R1) Ocupación del hueco (h·l² restricciones)
Σ_{k∈K} x[k,i,j,m] = y[i,j,m] ∀ i ∈ I, j ∈ J, m ∈ M, con 0 ≤ y[i,j,m] ≤ 1
Sustituye a la igualdad «= 1» de la parte 2, que con n < h·l² haría el problema infactible siempre. La igualdad define y, y es la cota y ≤ 1 la que impone como máximo una caja por hueco; y = 0 es el hueco vacío.

(R2) Cada caja, un hueco (n restricciones)
Σ_{i∈I} Σ_{j∈J} Σ_{m∈M} x[k,i,j,m] = 1 ∀ k ∈ K
Sigue siendo igualdad: las n cajas hay que colocarlas todas

(R3) Orden por prioridad (h·l(l−1) restricciones)
$Q[i,j,m] \geq Q[i+1,j,m] \quad \forall\ i \in \{1..l-1\},\ j \in J,\ m \in M$
Produce toda la geometría de los huecos vacíos: los huecos ocupados forman un bloque pegado al acceso y los vacíos quedan al fondo

(R4) Prioridad de hueco (h·l² restricciones)
$P[i,j,m] = \Sigma_{k \in K}\ p_k \cdot x[k,i,j,m] \quad \forall\ i \in I,\ j \in J,\ m \in M$

(R5) Peso de hueco (h·l² restricciones)
$W[i,j,m] = \Sigma_{k \in K}\ w_k \cdot x[k,i,j,m] \quad \forall\ i \in I,\ j \in J,\ m \in M$

(R6) Capacidad de hueco (h·l² restricciones)
$\Gamma[i,j,m] = \Sigma_{k \in K}\ \gamma_k \cdot x[k,i,j,m] \quad \forall\ i \in I,\ j \in J,\ m \in M$

(R7) Resistencia acumulada (l²(h−1) restricciones)
$\Sigma_{m'=m+1..h}\ W[i,j,m'] \leq \Gamma[i,j,m] \quad \forall\ i \in I,\ j \in J,\ m \in \{1..h-1\}$

Con huecos vacíos sigue siendo consistente. Si (i,j,m) está vacío entonces Γ 0, y R9 garantiza que todo lo que tiene encima está vacío 0<= 0

(R8) Prioridad de orden del hueco vacío (h·l² restricciones)

$Q[i,j,m] = P[i,j,m] + p^{\infty} \cdot (1 - y[i,j,m]) \quad \forall\ i \in I,\ j \in J,\ m \in M$

Traduce la propiedad prioridad = +∞. Si el hueco está ocupado (y = 1) queda Q = P = p_k; si está vacío (y = 0), P = 0 y queda Q = p^∞. Es lineal porque p^∞ es un parámetro y no una variable.

(R9) Apoyo entre niveles (l²(h−1) restricciones)
$y[i,j,m+1] \leq y[i,j,m] \quad \forall\ i \in I,\ j \in J,\ m \in \{1..h-1\}$
Restricción que hace que encima de un hueco vacío no pueda ir ninguna caja, pero sí otro hueco vacío. Usamos ≤ y no = porque la igualdad prohibiría también el caso vacío sobre ocupado, que sí que se puede. El enunciado admite w_k = 0, y con una caja de peso nulo R7 se satisfaría con esa caja flotando sobre un hueco vacío, por lo que no choca con R7

(R10) Coste imputado al hueco (h·l(l−1) restricciones)
$c[i,j,m] \geq \Sigma_{i'=i+1..l}\ P[i',j,m] - \beta_i \cdot (1 - y[i,j,m]) \quad \forall\ i \in \{1..l-1\},\ j \in J,\ m \in M$

El coste de la posición vacía es 0. Si el hueco está ocupado (y = 1) queda c ≥ la suma de las prioridades de delante; si está vacío (y = 0) el lado derecho es ≤ 0, porque la suma tiene l−i sumandos acotados cada uno por p^max, y con c ≥ 0 resulta c = 0. Se usa ≥ y no igualdad porque con igualdad el hueco vacío forzaría c a un valor negativo, infactible;

En total $5h \cdot l^2 + n + 2h \cdot l(l-1) + 2l^2(h-1)$ restricciones.

4.5. Decisiones de diseño

Separar P (coste) de Q (orden). El hueco vacío debe valer +∞ en R3 y 0 en el objetivo, y una sola variable no puede hacer ambas cosas: con P en R3 el vacío, al tener P = 0, se pone como la mejor prioridad y los huecos se irían hacia el acceso; y con p^∞ en el objetivo el coste se pone alto. poner P

mediante un indicador no es lineal, pero Q = P + p^∞(1−y) resuelve los dos usos con una variable y una restricción por hueco.

Elección de p^∞ = p^max + 1. No podíamos poner literalmente infinito, un número mayor a la mayor prioridad hace de nuestro “infinito”.

Por qué hace falta la variable c. La forma compacta Σ(i−1)·p_k·x de la parte 2 solo es correcta con el palet lleno: si n < h·l², algunos de los huecos que el coeficiente (i−1) cuenta como que tienen coste están vacíos aunque no deba contarse el coste 0 como coste.

Alternativa descartada. Intentar recrear el palet lleno con cajas inventadas era una posibilidad pero no sirve porque se seguiría tomando el coste de estas cajas inventadas que en realidad son huecos y no debería ser así.

5. Parte 4: varios palets

Esta parte añade que hay posibilidad ilimitada de palets, cada uno con un máximo de h niveles, y un coste fijo P por cada palet montado. El coste total es la suma del coste de disposición de las cajas de cada palet más el coste de preparación de cada palet utilizado.

Sigue siendo un problema de programación lineal entera mixta, con la familia de asignación x binaria y el resto de variables continuas, determinadas por sus igualdades de definición. A la estructura de la parte 3 se añade una decisión de apertura por pallet, hay que decidir cuántos palets se montan y cómo se reparte la carga entre ellos.

5.1. Conjuntos y parámetros

Los conjuntos y parámetros son los de la parte 3, más los siguientes:

S = {1, …, q}: palets disponibles
κ ≥ 0: coste de montar un palet
q = ⌈n / l⌉: número de palets que basta considerar

El enunciado llama P a este coste, pero P es ya la variable de prioridad del hueco, de modo que lo denotamos κ para evitar la colisión.

La disponibilidad de palets es ilimitada, pero el modelo necesita un conjunto finito. No basta con ⌈n/l²⌉, el número de palets en los que las cajas caben: esa cota da factibilidad pero no optimalidad, porque al óptimo puede convenirle abrir palets de más, ya que repartir las cajas deja a más de ellas en la fila de acceso sin pagar coste de disposición. La cota correcta se apoya en el coste: con ⌈n/l⌉ palets caben l cajas por palet en la fila de acceso del nivel 1, sin apilar y por tanto sin que intervengan las capacidades, con coste de disposición 0 y total κ·⌈n/l⌉; como toda solución con p palets cuesta al menos κ·p, ninguna con más de ⌈n/l⌉ puede ser óptima. Sigue siendo más fina que la cota trivial q = n, que surge de dar un palet a cada caja: en la instancia del enunciado da 3 en lugar de 6.

Mantener q ajustado no solo reduce el número de variables: como los palets son intercambiables entre sí, también recorta el espacio de soluciones equivalentes que el branch and bound tiene que descartar.

5.2. Variables de decisión

Las de la parte 3 con el índice de palet s añadido —x, y, P, Q, W, Γ y c— más una nueva:

V8: abierto[s] ∈ [0,1] ∀ s ∈ S
Vale 1 si el palet s se monta y 0 si no se usa. q variables.
En total $h \cdot l^2 \cdot q(n+5) + h \cdot l(l-1) \cdot q + q$ variables, de las cuales $n \cdot h \cdot l^2 \cdot q$ son binarias.

5.3. Función objetivo

$\min z = \sum_{i<l} \sum_{j \in J} \sum_{m \in M} \sum_{s \in S} c[i,j,m,s] + \kappa \cdot \sum_{s \in S} abierto[s]$
se pide el coste total y no el medio, de modo que desaparece el factor 1/n. Los dos sumandos son el coste de disposición de todas las cajas, idéntico al de la parte 3 pero agregado sobre los palets, y el coste de preparación de los palets efectivamente montados.

5.4. Restricciones

R1 a R10 son las de la parte 3 con el índice de palet añadido. Se aplican dentro de cada palet por separado, porque las torres, el orden por prioridad y los bloqueos no cruzan de un palet a otro.

(R1) Ocupación del hueco ($h \cdot l^2 \cdot q$ restricciones)
$\sum_{k \in K} x[k,i,j,m,s] = y[i,j,m,s] \quad \forall\ i \in I, j \in J, m \in M, s \in S$, con $0 \le y[i,j,m,s] \le 1$
Una caja o ninguna por hueco; y = 0 es el hueco vacío.

(R2) Cada caja, un hueco (n restricciones)
$\sum_{i \in I} \sum_{j \in J} \sum_{m \in M} \sum_{s \in S} x[k,i,j,m,s] = 1 \quad \forall\ k \in K$
Única restricción que agrega sobre s: cada caja va a un solo hueco de un solo palet.

(R3) Orden por prioridad ($h \cdot l(l-1) \cdot q$ restricciones)
$Q[i,j,m,s] \ge Q[i+1,j,m,s] \quad \forall\ i \in \{1..l-1\}, j \in J, m \in M, s \in S$
Orden dentro de cada columna, nivel y palet.

(R4) Prioridad de hueco ($h \cdot l^2 \cdot q$ restricciones)
$P[i,j,m,s] = \sum_{k \in K} p_k \cdot x[k,i,j,m,s] \quad \forall\ i \in I, j \in J, m \in M, s \in S$

(R5) Peso de hueco ($h \cdot l^2 \cdot q$ restricciones)
$W[i,j,m,s] = \sum_{k \in K} \omega_k \cdot x[k,i,j,m,s] \quad \forall\ i \in I, j \in J, m \in M, s \in S$

(R6) Capacidad de hueco ($h \cdot l^2 \cdot q$ restricciones)
$\Gamma[i,j,m,s] = \sum_{k \in K} \gamma_k \cdot x[k,i,j,m,s] \quad \forall\ i \in I, j \in J, m \in M, s \in S$
R4, R5 y R6 definen las tres magnitudes de la caja alojada; dan 0 si el hueco está vacío.

(R7) Resistencia acumulada ($l^2(h-1) \cdot q$ restricciones)
$\sum_{m'=m+1..h} W[i,j,m',s] \le \Gamma[i,j,m,s] \quad \forall\ i \in I, j \in J, m \in \{1..h-1\}, s \in S$
Resistencia dentro de cada torre de cada palet.

(R8) Prioridad de orden del hueco vacío ($h \cdot l^2 \cdot q$ restricciones)
$Q[i,j,m,s] = P[i,j,m,s] + p^{\wedge}\infty \cdot (1 - y[i,j,m,s]) \quad \forall\ i \in I, j \in J, m \in M, s \in S$

(R9) Apoyo entre niveles ($l^2(h-1) \cdot q$ restricciones)
$y[i,j,m+1,s] \le y[i,j,m,s] \quad \forall\ i \in I, j \in J, m \in \{1..h-1\}, s \in S$

(R10) Coste imputado al hueco ($h \cdot l(l-1) \cdot q$ restricciones)
$c[i,j,m,s] \ge \sum_{i'=i+1..l} P[i',j,m,s] - \beta_i \cdot (1 - y[i,j,m,s]) \quad \forall\ i \in \{1..l-1\}, j \in J, m \in M, s \in S$

(R11) Apertura de palet ($h \cdot l^2 \cdot q$ restricciones)
$y[i,j,m,s] \leq abierto[s] \quad \forall\ i \in I, j \in J, m \in M, s \in S$

Enlaza ocupación y apertura: si algún hueco del palet s aloja una caja, el palet hay que montarlo y se paga su coste. Se escribe como desigualdad y no como igualdad porque un palet abierto no tiene por qué estar lleno

En total $6h \cdot l^2 \cdot q + n + 2h \cdot l(l-1) \cdot q + 2l^2(h-1) \cdot q$ restricciones.

5.5. Decisiones de diseño

La variable de apertura continua en [0,1]. Por el mismo motivo que y en la parte 3: R11 la empuja hacia arriba hasta el máximo de las y de su palet, que son enteras, y el término $\kappa \cdot$abierto del objetivo la empuja hacia abajo hasta ese mismo valor. Queda por tanto en 0 o 1 en todo óptimo sin declararla binaria, lo que evita q variables sobre las que ramificar. En el caso límite $\kappa = 0$ el objetivo no ejerce esa presión y la variable queda indeterminada en los palets vacíos, pero su valor no interviene entonces en el coste, de modo que sigue existiendo un óptimo con $abierto[s] \in \{0,1\}$.

6. Parte 5: Análisis de resultados

6.1. Metodología y tamaño de los modelos

Todas las medidas se han obtenido ejecutando los scripts de cada parte sobre instancias generadas variando un parámetro cada vez, con prioridades y pesos aleatorios en rangos fijos y capacidades holgadas salvo donde se indique lo contrario. El número de restricciones que imprime cada script descuenta la fila de la función objetivo, que GLPK contabiliza pero no es una restricción del modelo.

Elemento	Parte 1	Parte 2	Parte 3	Parte 4
Variables binarias x	$n \cdot l^2$	$n \cdot h \cdot l^2$	$n \cdot h \cdot l^2$	$n \cdot h \cdot l^2 \cdot q$
Variables auxiliares	l^2	$3h \cdot l^2$	$5h \cdot l^2 + h \cdot l(l-1)$	$(5h \cdot l^2 + h \cdot l(l-1)) \cdot q + q$
Total variables	$l^2(n+1)$	$h \cdot l^2(n+3)$	$h \cdot l^2(n+5) + h \cdot l(l-1)$	$(h \cdot l^2(n+5) + h \cdot l(l-1)) \cdot q + q$
Total restricciones	$2l^2 + n + l(l-1)$	$5h \cdot l^2 + h \cdot l(l-1) + l^2(h-1)$	$5h \cdot l^2 + n + 2h \cdot l(l-1) + 2l^2(h-1)$	$6h \cdot l^2 \cdot q + n + 2h \cdot l(l-1) \cdot q + 2l^2(h-1) \cdot q$

Las mediciones confirman las fórmulas en todas las instancias probadas. Conviene destacar que el número de variables no predice por sí solo la dificultad: la instancia mayor de la parte 1, con 1332 variables, se resuelve en 0,05 s, mientras que una de la parte 3 con 264 tarda bastante más

6.2. Análisis de la parte 1

¿Se puede resolver sin Programación Lineal? Sí. El coste admite la forma $\Sigma(i-1) \cdot p_k \cdot x$, de modo que solo depende de la fila en que acaba cada caja y no de su columna: basta ordenar las prioridades de forma descendente y rellenar el palet por filas, poniendo las peores al fondo, con coste n log n. Lo hemos comprobado comparando el óptimo del modelo con el de esa ordenación:

l	óptimo del modelo	ordenación
3	7,222222	7,222222

4	8,937500	8,937500
5	12,560000	12,560000

¿Puede ser infactible o no acotada? No, en ningún caso. El palet se rellena siempre completamente con $n = l^2$ cajas, de modo que cualquier ordenación por filas es una disposición factible; y la función objetivo es una suma de prioridades no negativas. Tiene límite 0, acotada

¿Puede haber más de una solución óptima? Sí, por tres vías. Las permutaciones de columnas, porque el índice j no es importante en todas las restricciones y en el objetivo, lo que produce hasta l! óptimos equivalentes. Los reordenamientos dentro de una misma fila, que como se ha visto no alteran el coste. Y los empates entre cajas de prioridad idéntica, que son intercambiables entre sí.

Tamaño y tiempo. El número de variables crece como l^4, pero el tiempo no lo acusa: de l = 2 a l = 6, con 20 y 1332 variables respectivamente, todas las instancias se resuelven entre 0,02 y 0,05 s. La razón es que, como el objetivo por sí solo induce la ordenación óptima, R3 no elimina ninguna solución óptima y la relajación lineal resulta muy ajustada, de modo que el branch and bound apenas tiene que ramificar.

6.3. Análisis de la parte 2

¿Se puede resolver sin Programación Lineal? No. La estructura del coste es la misma que en la parte 1, pero las capacidades acoplan los niveles de cada torre y la ordenación deja de bastar. Se comprueba fijando una instancia y estrechando solo las capacidades:

capacidad	óptimo
999 (holgadas)	1,250000
30 (medias)	1,250000
14 (ajustadas)	1,500000
0 (nulas)	infactible

al estrechar las capacidades del óptimo sube a 1,50: la disposición que la ordenación elegiría es infactible, y R7 pasa de redundante a activa.

¿Puede ser infactible o no acotada? Acotada siempre lo está, por el mismo argumento de la parte 1. Pero a diferencia de ella sí puede ser infactible: el palet va lleno por definición, con $n = h \cdot l^2$ cajas que hay que apilar en h niveles, y si las capacidades no lo permiten no hay solución. El caso extremo medido es $\gamma_k = 0$ para toda caja con pesos estrictamente positivos, que impide ocupar cualquier nivel por encima del primero mientras R2 exige colocar las $h \cdot l^2$ cajas.

¿Puede haber más de una solución óptima? Sí, por las mismas tres vías de la parte 1, aplicadas ahora dentro de cada nivel. Se añade que, cuando las capacidades lo permiten, dos niveles completos pueden intercambiarse entre sí sin alterar el coste, ya que este no depende del nivel.

Tamaño y tiempo. Las variables crecen como $h^2 \cdot l^4$, y la mayor instancia probada, con l = 3 y h = 3, llega a 810 variables y 171 restricciones resolviéndose aun así en 0,03 s. El palet lleno sigue siendo la clave: al no haber huecos vacíos, cada posición aloja exactamente una caja y el problema es una asignación pura, sin que el modelo tenga que decidir cuántas cajas van en cada columna.

6.4. Análisis de la parte 3

¿Se puede resolver sin Programación Lineal? No, y por un motivo adicional al de la parte 2: con $n < h \cdot l^2$ hay que decidir además cómo repartir las cajas entre las columnas y hasta qué altura levantar

cada torre. Ese reparto es precisamente lo que determina el coste y no se deduce de la ordenación de prioridades.

¿Puede ser infactible o no acotada? Acotada siempre. Infactible sí, cuando $n > l^2$ obliga a apilar y las capacidades no lo permiten. La predicción del modelo es que con $y_k = 0$ y pesos positivos caben a lo sumo l^2 cajas; lo hemos medido con $l = 2$ y $h = 2$:

n	resultado
3	factible, z = 1,666667
4	factible, z = 4,250000
5	infactible
6	infactible

El corte está exactamente en $n = l^2 = 4$, como se esperaba. Esto es lo que motiva la parte 4.

¿Puede haber más de una solución óptima? Sí, por las vías anteriores más una nueva: la elección de qué huecos quedan vacíos. Cuando varias configuraciones del bloque ocupado dan el mismo coste, todas son óptimas.

Cota inferior del coste. El coste solo puede ser 0 si $n \leq l \cdot h$, porque los únicos huecos sin coste son los de la fila de acceso de cada par columna-nivel, que son exactamente $l \cdot h$. Las mediciones lo confirman en los dos sentidos: con $l = 2$ y $h = 3$ ($l \cdot h = 6$) las instancias de $n = 4$ y $n = 6$ dan óptimo 0, mientras que la de $n = 9$ da 1,444444; y con $l = 3$ y $h = 2$ ($l \cdot h = 6$ de nuevo) la de $n = 6$ da 0 y la de $n = 9$ da 0,777778.

Tamaño y tiempo. Es el resultado más llamativo del análisis: el tiempo no crece con el tamaño, sino que alcanza su máximo en las instancias intermedias. Con $l = 3$ y $h = 2$:

n	variables	tiempo
6	210	0,02 s
9	264	37,25 s
14	354	> 90 s
18 (lleno)	426	0,03 s

La instancia mayor, con el palet completo, es más de mil veces más rápida que una con cuatro cajas menos. La explicación es estructural y tiene dos extremos. Cuando $n \leq l \cdot h$, todas las cajas caben en posiciones de coste nulo, el óptimo es 0 y la relajación lineal lo alcanza de inmediato, de modo que no hay nada que ramificar. Cuando $n = h \cdot l^2$ el palet va lleno, la ocupación deja de ser una decisión, todas las y valen 1 y el modelo degenera literalmente en el de la parte 2. La dificultad vive entre ambos extremos, que es donde el modelo debe decidir simultáneamente cuántas cajas van en cada columna, hasta qué altura y cuáles quedan al fondo.

A ello se suman los dos factores que ya se discutieron al justificar el modelo: la simetría entre columnas, que multiplica por hasta $l!$ los nodos equivalentes y es más dañina aquí que en las partes anteriores porque el reparto entre columnas es justamente la decisión central; y la debilidad de la relajación lineal inducida por la constante grande de R10, que con y fraccionaria permite abaratar el coste y deja la cota inferior lejos del óptimo entero, con lo que la poda pierde eficacia. Dentro del límite de 90 s se resolvieron todas las instancias probadas salvo la de $n = 14$ con $l = 3$ y $h = 2$, lo que confirma que el tamaño no es el factor determinante.

6.5. Análisis de la parte 4

¿Se puede resolver sin Programación Lineal? No. Al problema de la parte 3 se añade decidir cuántos palets se montan y qué cajas van en cada uno, que es una estructura de empaquetado en contenedores (bin packing), aquí el problema no es en si el tamaño sino las capacidades.

¿Puede ser infactible o no acotada? Ni lo uno ni lo otro, y en esto se diferencia de la parte 3. Por el argumento de la cota q, la disposición que coloca l cajas por palet en la fila de acceso del nivel 1 de los q palets satisface todas las restricciones sea cual sea el conjunto de pesos y capacidades, porque al no apilar ninguna caja las de resistencia se cumplen de forma trivial. Lo que en la parte 3 era infactibilidad aquí se convierte en coste: cuando las capacidades impiden apilar, el modelo abre más palets.

¿Puede haber más de una solución óptima? Sí, y la multiplicidad es la mayor de las cuatro partes. A las vías de la parte 3 se añade la permutación de palets completos: como cada palet replica internamente la simetría de columnas, el número de reetiquetados de una misma solución crece como q!·(l!)^q. Y aparece una fuente cualitativamente distinta: en el valor de κ que iguala las dos estrategias coexisten óptimos con distinto número de palets abiertos, como se ve a continuación.

El papel de κ. Fijada una instancia con l = 2, h = 2 y n = 6, y variando solo el coste de montaje:

κ	coste total	descomposición
0	0	2 palets, disposición 0
1	2	2 palets, disposición 0
3	6	2 palets, disposición 0
10	17	1 palet, disposición 7
50	57	1 palet, disposición 7
7	14	2 palet disposición 0 o 1 palet con disposición 7

El parámetro gobierna la tensión del problema: con κ bajo compensa repartir las cajas entre más palets, porque así más quedan en la fila de acceso y no pagan coste de disposición; con κ alto compensa concentrarlas y aceptar que algunas queden al fondo., El punto de equilibrio de esta instancia está exactamente en κ = 7, donde las dos estrategias cuestan 14 y son óptimas a la vez: repartir en dos palets cuesta 2κ = 14 con disposición 0, y concentrar en uno cuesta κ + 7 = 14. Por debajo de κ = 7 compensa repartir; por encima, concentrar..

Tamaño y tiempo. Es la parte que peor escala, porque el modelo de la parte 3 se replica q veces y la simetría entre palets se suma a la de columnas.

l	h	n	q	variables	tiempo
2	2	4	2	154	0,03 s
2	2	6	3	279	0,14 s
2	2	8	4	436	6,95 s
2	3	9	5	—	> 90 s
3	2	9	3	—	> 90 s

El mayor tamaño resuelto dentro del límite ha sido n = 8 con l = 2 y h = 2. Obsérvese que n interviene dos veces, en el número de variables por palet y en el propio q, de modo que el crecimiento es cuadrático en n a parámetros fijos.

6.6. Conclusiones del análisis

Las cuatro partes forman una progresión en la que cada una añade una estructura que la anterior no tenía, y el salto de dificultad no está donde el tamaño del modelo sugeriría. La parte 1 es resoluble en tiempo n log n sin recurrir a Programación Lineal. La parte 2 ya no lo es, porque las capacidades acoplan los niveles de cada torre, pero sigue siendo fácil de resolver porque el palet lleno convierte el problema en una asignación pura. El salto real está en la parte 3, donde la ocupación pasa a ser una decisión: aparece la infactibilidad, aparece la constante grande que debilita la relajación lineal, y el tiempo deja de ser monótono en el tamaño para concentrarse en las instancias intermedias. La parte 4 hereda todo eso y lo multiplica por el número de palets, añadiendo una simetría factorial que la convierte en la única parte donde instancias de nueve cajas quedan fuera de alcance.

De las decisiones de modelado tomadas, las que más han influido en el tiempo de resolución han sido declarar continuas todas las variables auxiliares, que reduce a la familia x el conjunto sobre el que ramificar, y acotar el número de palets de la parte 4 por $\lceil n/l \rceil$ en lugar de por n, que divide entre dos el tamaño del modelo en las instancias probadas.

7. Conclusiones

En esta práctica, un problema de disposición espacial ha sido progresivamente solucionado, modelado y optimizado usando programación lineal y el solver glpk. En cuatro partes el modelo ha evolucionado de un simple problema bidimensional a un problema análogo a bin packing en tres dimensiones con múltiples contenedores y costes fijos.

En cada parte se presentan desafíos específicos de modelado. En la segunda, la mayor dificultad residía en formular algebraicamente las restricciones de apilamiento, asegurando que el peso acumulado no excediera la capacidad de la caja en la base sin perder la linealidad del modelo. La tercera fue la que más dificultades ocasionó, ya que introdujo la posibilidad de palets incompletos donde el desafío era gestionar la asimetría de huecos vacíos, que tienen cero capacidad pero infinita prioridad, forzando el diseño a un sistema de prioridad dual, con P para el costo y Q para el ordenamiento geométrico, usando Big M. Finalmente la parte 4 introdujo el desafío de la explosión combinatoria y extrema simetría a la hora de permutar cajas de diferentes palets.

El desarrollo de estos modelos nos ha permitido aprender que la fundamental diferencia entre un modelo que es correcto y otro que es computacionalmente viable reside en las decisiones de diseño. Estrategias como la relajación continua de variables de ocupación y apertura, y la acotación del número máximo de pallets necesarios, entre otras, han demostrado ser esenciales. Estas decisiones no solo preservan la requerida linealidad, sino que también reducen drásticamente el espacio de búsqueda y el número de variables binarias sobre las que el algoritmo de branch and bound debe ramificar. Definitivamente, esta práctica demuestra que un planteamiento correcto es tan importante como el propio algoritmo de resolución.